

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    accuracy_score
)

# -----
# Load Dataset
# -----

df = pd.read_csv("loan_data.csv")

# -----
# Drop Loan_ID Column
# -----

if "Loan_ID" in df.columns:
    df.drop("Loan_ID", axis=1, inplace=True)
```

```
# -----
```

```
# Handle Missing Numerical Values
```

```
# -----
```

```
num_cols = [
```

```
    "ApplicantIncome",
```

```
    "CoapplicantIncome",
```

```
    "LoanAmount",
```

```
    "Loan_Amount_Term",
```

```
    "Credit_History"
```

```
]
```

```
for col in num_cols:
```

```
    df[col] = df[col].fillna(df[col].mean())
```

```
# -----
```

```
# Handle Missing Categorical Values
```

```
# -----
```

```
cat_cols = [
```

```
    "Gender",
```

```
    "Married",
```

```
    "Dependents",
```

```
    "Education",
```

```
    "Self_Employed",
```

```

    "Property_Area",
    "Loan_Status"
]

for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

# -----
# Encode Categorical Columns
# -----

encoder = LabelEncoder()

for col in cat_cols:
    df[col] = encoder.fit_transform(df[col])

# -----
# Separate Features and Target
# -----

X = df.drop("Loan_Status", axis=1)
y = df["Loan_Status"]

# -----
# Split Dataset
# -----

```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.20,  
    random_state=42,  
    stratify=y  
)
```

```
# -----  
# Decision Tree Function  
# -----
```

```
def decisionTree(X_train, X_test, y_train, y_test):
```

```
    model = DecisionTreeClassifier(random_state=42)
```

```
    model.fit(X_train, y_train)
```

```
    y_pred = model.predict(X_test)
```

```
    train_accuracy = accuracy_score(y_train, model.predict(X_train))
```

```
    test_accuracy = accuracy_score(y_test, y_pred)
```

```
    print("=" * 50)
```

```
    print("Decision Tree Model")
```

```
print("=" * 50)
```

```
print("\nTraining Accuracy :", round(train_accuracy * 100, 2), "%")
```

```
print("Testing Accuracy :", round(test_accuracy * 100, 2), "%")
```

```
print("\nConfusion Matrix")
```

```
print(confusion_matrix(y_test, y_pred))
```

```
print("\nClassification Report")
```

```
print(classification_report(y_test, y_pred))
```

```
# -----
```

```
# Call Function
```

```
# -----
```

```
decisionTree(X_train, X_test, y_train, y_test)
```